

Q1. (a)

(i) (1 mark)

$$\Omega(n^{2025})$$

(ii) (1 mark)

$$\Omega(2^n)$$

(iii) (1 mark)

$$\Omega(n!)$$

(iv) (1 mark)

$$\Omega(n^2)$$

(v) (1 mark)

$$\Omega(\log n)$$

(vi) (1 mark)

$$\Omega(\log^{32} n)$$

(b) (6 marks)

Proof: There exist  $c = 2$ ,  $n_0 = 1$  such that for all  $n > n_0$ ,  
 $n^{0.1} \geq 1$   
 $2n^2 \leq 2n^{2.1}$   
 $2n^2 \leq cn^{2.1}$

(c) (6 marks)

We need to prove that  $[\exists c, n_0 \bullet \forall n > n_0 \bullet 2n^2 \geq cn^{2.1}]$  is wrong.  
It is equivalent to proving  $\forall c, n_0 \bullet \exists n > n_0 \bullet 2n^2 < cn^{2.1}$ .  
Proof:  
For all  $c, n_0$ , there exists  $n = \max(\lceil (2/c)^{10} \rceil, n_0) + 1 > n_0$  such that  
 $n > (2/c)^{10}$   
 $cn^{0.1} > 2$   
 $2n^2 < cn^{2.1}$

(d) (7 marks)

Proof: first, we prove  $O(n) \subset O(3n)$ .  
Let  $f(n) = O(n)$ .  
Then, we have  $\exists c_0, n_0 \bullet \forall n > n_0 \bullet f(n) \leq c_0 n$ .  
 $\exists c_1 = c_0/3, n_1 = n_0 \bullet \forall n > n_1 \bullet$   
 $n > n_0$   
 $f(n) \leq c_0 n = c_1(3n)$   
Rewriting,  $\exists c_1, n_1 \bullet \forall n > n_1 \bullet f(n) \leq c_1 3n$ .  
Then,  $f(n) = O(3n)$ .  
  
Now, we prove  $O(3n) \subset O(n)$ .  
Let  $f(n) = O(3n)$ .  
Then, we have  $\exists c_0, n_0 \bullet \forall n > n_0 \bullet f(n) \leq c_0(3n)$ .  
 $\exists c_1 = 3c_0, n_1 = n_0 \bullet \forall n > n_1 \bullet$   
 $n > n_0$   
 $f(n) \leq c_0(3n) = c_1 n$   
Rewriting,  $\exists c_1, n_1 \bullet \forall n > n_1 \bullet f(n) \leq c_1 n$ .

$$f(n) = O(n)$$

Done.

Q2. (a) (i) (3 marks)

$$\begin{aligned} \exists k = 0 \bullet \\ n^{\log_3 3} \log^k n &= n^{(1)} \log^0 n = n \\ \text{Therefore, } n &= \Theta(n^{\log_3 3} \log^k n) \end{aligned}$$

(ii) (3 marks)

$$\begin{aligned} a = 3, b=3, f(n) &= n \\ \text{By (i), } \exists k = 0 \bullet f(n) &= \Theta(n^{\log_b a} \log^k n) \\ \text{Case 2 applies.} \\ T(n) &= \Theta(n^{\log_b a} \log^{k+1} n) \\ T(n) &= \Theta(n \log n) \end{aligned}$$

(b) (i) (3 marks)

$$\begin{aligned} \exists \epsilon = 1 \bullet \\ n^2 &= n^{\log_3 3 + \epsilon} \\ n^2 &= \Omega(n^{\log_3 3 + \epsilon}). \end{aligned}$$

(ii) (3 marks)

$$\begin{aligned} \exists c = 1/3 < 1 \bullet \\ 3f(n/3) &= 3(n/3)^2 = (1/3)n^2 = cn^2 = c f(n) \end{aligned}$$

(iii) (3 marks)

$$\begin{aligned} a = 3, b=3, f(n) &= n^2 \\ \text{By (i),} \\ \exists \epsilon = 1 \bullet \\ f(n) &= n^2 = \Omega(n^{\log_3 3 + \epsilon}) = \Omega(n^{\log_b a + \epsilon}). \\ \text{Regularity checking} \\ \text{By (ii),} \\ \exists c = 1/3 < 1 \bullet \\ \text{Therefore, } af(n/b) &= 3f(n/3) \leq cf(n). \\ \text{Case 3 applies.} \\ T(n) &= \Theta(f(n)) \\ T(n) &= \Theta(n^2) \end{aligned}$$

(c) (i) (4 marks)

$$\begin{aligned} T(n) &= 2T(n/3) + n \\ &= 2[2T(n/3^2) + (n/3)] + n \\ &= 2^2 T(n/3^2) + n[1 + (2/3)] \\ &= 2^2 [2T(n/3^3) + (n/3^2)] + 2n \\ &= 2^3 T(n/3^3) + n[1 + (2/3) + (2/3)^2] \\ &= 3^k T(n/3^k) + n[1 + (2/3) + (2/3)^2 + \dots + (2/3)^{k-1}] \\ &= nT(1) + n[1 - (2/3)^k] / [1 - (2/3)], \text{ setting } n = 3^k, k = \log_3 n \\ &= O(n) \end{aligned}$$

(ii) (4 marks)

Claim:  $T(n) < c n$ ,  $c > 3$ ,  $n > 2$   
Proof:

$$\begin{aligned}
T(n) &= 2T(n/3) + n \\
&< 2[c(n/3)] + n \\
&= (2/3)cn + n \\
&< cn, \text{ since } c > 3.
\end{aligned}$$

(d) (2 marks)

$T(n)$  should have format  $aT(n/b) + f(n)$ , where  $a \geq 1$ ,  $b > 1$ ,  $f(n)$  is positive. Now  $n-1$  is not a fraction of  $n$ .

Q3. (a) (i) (5 marks)

For a binary tree  $T$ , if  $T$  is empty, then the height of  $T$  is 0; else the height of  $T$  is  $\max(\text{height of } T_L, \text{height of } T_R) + 1$ , where  $T_L$  and  $T_R$  are the left and right subtrees, respectively.

(ii) (4 marks)

```

Algorithm height(binary tree T)
If T is empty
    return 0;
else
     $h_L = \text{height}(T_L);$ 
     $h_R = \text{height}(T_R);$ 
    return  $\max(h_L, h_R) + 1;$ 

```

(iii) (4 marks)

Let  $G(n)$  be the time complexity of height(binary tree  $T$ ), where  $n$  is the number of nodes in  $T$ .  
 For non-empty  $T$ , let  $n_L$  and  $n_R$  be the number of nodes in the left and right subtrees, respectively.  
 Then,  $G(n) = G(n_L) + G(n_R) + 1$   
 Claim  $G(n) < cn$ ,  $c > 1$ .  
 $G(n) = G(n_L) + G(n_R) + 1$   
 $< c n_L + c n_R + 1$   
 $< c n_L + c n_R + c$   
 $= cn$

(b) (10 marks)

Base case:  $n=1$ .  
 For a single-node tree, the two pointers of the node must be assigned to null. There are two null pointers.  
 The statement is true for  $n=1$ .  
 Assume the statement is true for  $n=k$ , where  $k$  is any positive integer.  
 Consider the statement for  $n=k+1$ .  
 Given any  $(k+1)$ -node binary tree, we take away a node  $X$  which does not have any child. The pointer  $p$  which pointed to  $X$  becomes null. The tree now has  $k$  nodes. The number of null pointers is  $k+1$ , by the inductive assumption.  
 Now, we put node  $x$  back to its original position. The pointer  $p$ , which was null, is now pointing to  $x$ . Counting the two null pointers of  $x$ , the whole tree will have  $k+1 - 1 + 2 = k+2$  pointers.  
 So, there are  $k+2$  null pointers in the original tree.  
 The statement is true for  $n=k+1$ .  
 Result from induction.

$$n(N - 1) + 1$$
$$\begin{aligned} \text{sum}(i,j) &= 0, \text{ for } j < i; \\ \text{sum}(i,j) &= x_i, \text{ for } j = i; \end{aligned}$$

```

Algorithm sum(i,j)
  If j < i
    return 0;
  If j = i
    return X[i];
  If j > i
    t1 = sum(i+2,j);
    t2 = sum(i+3,j);
    return max(X[i] + t1, X[i+1] + t2);

```

|     |     |   |   |   |    |
|-----|-----|---|---|---|----|
|     | j=1 | 2 | 3 | 4 | 5  |
| i=1 | 1   | 4 | 4 | 8 | 12 |
| 2   |     | 4 | 4 | 8 | 12 |
| 3   |     |   | 1 | 4 | 9  |
| 4   |     |   |   | 4 | 8  |
| 5   |     |   |   |   | 8  |

The lower triangle is all zero.

```
Construct a 2D array S[1..N, 1..N];  
// S[i,j] will store sum(i,j) eventually.  
  
For i = 2 to N  
    For j = 1 to i-1  
        S[i,j] = 0;  
For i = 1 to N  
    S[i,i] = X[i];  
  
For d = 2 to N  
    For j = d to N  
        i = j - d + 1;  
        if i+2 <= N  
            t1 = S[i+2,j];  
        else  
            t1 = 0;  
        if i+3 <= N  
            t2 = S[i+3,j];  
        else  
            t2 = 0;  
        S[i,j]=max(X[i]+t1, X[i+1]+t2);  
Return S[1,N];
```

Q5. (a) (i) (4 marks)

Vertex  $s$  is connected to all other  $n-1$  vertices.  
In the first iteration of the while loop, just before **DEQUEUE** is executed, all  $n-1$  vertices will be put into  $Q$ . Together with  $s$ , there are  $n$  vertices inside  $Q$ .

(ii) (4 marks)

The graph is a linear array of vertices. Vertex  $s$  is one of the two ending vertices.

At the beginning of the first iteration of the while loop, only vertex  $s$  is in  $Q$ , then the neighbour  $t$  of vertex  $s$  is stored into  $Q$ , and then **DEQUEUE** takes away vertex  $s$ , leaving vertex  $t$  in  $Q$ . Therefore, at the beginning of the second iteration, only one vertex is in  $Q$ . The same pattern continues, and there is exactly one element in  $Q$  at the beginning of an iteration.

(iii) (3 marks)

$n$ .  
Every iteration a single vertex will be dequeued, and a dequeued vertex cannot be enqueued again, because every vertex will not be enqueued again (given). So, the number of iterations is no more than  $n$ . If the number of iterations is less than  $n$ , then one vertex has not been dequeued, and due to the terminating condition of the while loop, the vertex has not been enqueued. It is a contradiction to given condition.

(b) (i) (3 marks)

$n$ .  
line (\*) is the only point to turn a vertex from white to gray. Since every vertex will be turned from white to gray for once, every vertex will go through line (\*) exactly once.

(ii) (3 marks)

$2e$ , which is also the sum of the nodes over all adjacency lists.  
For each vertex  $u$ , all the nodes in its adjacency list will be tested by the if-condition. So, summing up for all vertices, the number of executions of the if-condition =  $2e$ .

(iii) (4 marks)

From the algorithm, a vertex is turned from white to gray just before it is enqueued, and it is turned from gray to black right after it is dequeued. At the beginning (or the end) of each iteration, a vertex is gray iff it is inside the queue.

The condition [the colour of each vertex will be turned from white to gray, then from gray to black, and have no further changes] implies [every vertex will be enqueued into  $Q$  exactly once].

From (a)(iii), the number of iterations for the while loop is  $n$ . Together with the  $2e$  executions of the if-condition from (b)(ii), the time complexity is  $O(n + e)$ .

(c) (4 marks)

```
BFS( $G, s$ )
for each  $u \in V - \{s\}$  do
    color[ $u$ ]  $\leftarrow$  WHITE
color[ $s$ ]  $\leftarrow$  GRAY
parent[ $s$ ]  $\leftarrow$  NULL
 $Q \leftarrow \{s\}$ 
while  $Q \neq \emptyset$  do
     $u \leftarrow \text{head}[Q]$ 
    for each  $v$  in Adj[ $u$ ] do
        if color[ $v$ ] = WHITE then
            color[ $v$ ]  $\leftarrow$  GRAY          (*)
            parent[ $v$ ]  $\leftarrow u$ 
            ENQUEUE( $Q, v$ )
    DEQUEUE( $Q$ )
    color[ $u$ ]  $\leftarrow$  BLACK

Output parent[];
```